LonghornSilicon Documentation Library Proposal and Inventory

LonghornSilicon

Version library-0.1 2026-05-22

Abstract

LonghornSilicon now spans nineteen repositories and ships at least fourteen formal LaTeX documents — five published or draft papers, seven block-level specifications, and an emerging set of integration findings. The volume justifies a single canonical index. This document proposes folding the library role into the existing `LonghornSilicon/architecture` repository (rather than creating a new one), inventories every formal document across the organization, and lays out a four-step implementation plan to land a public-facing index within one work session.

1. Motivation

The organization's documentation footprint grew faster than its indexing scheme. A new collaborator (university partner, foundry contact, journal reviewer, future student) currently has to discover documents repo by repo. We have neither:

- A reading order across blocks (which paper covers the chip, which covers each block, which covers compression, which covers integration).
- A status register (which papers are submitted, which are drafts, which are superseded).
- A shared bibliography file so that all internal papers cite each other consistently.
- A public-facing entry point for an external visitor to grasp the program's scope without cloning each repo.

The cost of each missing piece is small in isolation but compounding. The Muse Semi meeting prep on 2026-05-21 is a recent example: the relevant artifacts (Sky130 sign-off, paper, ISA, integration audit, the TurboQuant+ draft) lived in five separate places, and the "what should I show them" question required reconstructing the map from memory rather than reading it off a page.

2. Inventory

The tables below cover every formal LaTeX-source-bearing document across the LonghornSilicon GitHub organization as of 2026-05-22. Each row shows the canonical source repository, the path within that repository, current status, and the document's role.

2.1 System-level / architecture

Document	Source	Status	Role
λ chip architecture	`architecture/paper/lambda.tex`	draft	full chip overview
Dataflow walkthrough	`architecture/dataflow_walkthrough.md`	living	block interaction explainer
Architecture status	`architecture/STATUS.md`	living	per-block development state
Floorplan visualisation	`architecture/floorplan.html`	living	visual aid

2.2 Block 1 — ACU (Attention Compute Unit)

Document	Source	Status	Role
Adaptive Precision Attention	`adaptive-precision-attention/paper/adaptive_precision_attention.tex`	draft	main block paper
Precision Controller ISA	`adaptive-precision-attention/docs/isa/precision_controller_isa.tex`	v0.1 stable	interface specification
MAC array design	`adaptive-precision-attention/docs/mac_array_design.tex`	v0.1 stable	sub-block design
Reference model API	`adaptive-precision-attention/docs/reference_model_api.tex`	v0.1 stable	compiler-facing API
SW overview	`adaptive-precision-attention/docs/sw_overview.tex`	v0.1 stable	compiler-team orientation
Meeting handout	`adaptive-precision-attention/docs/meeting_handout.tex`	current piece	one-pager (compiler meeting)

2.3 Block 2 — KV Cache Engine

Document	Source	Status	Role
KV Cache Engine ISA	`kv-cache-engine/docs/isa/kv_cache_engine_isa.tex`	v0.1 stable	interface specification
Reference model API	`kv-cache-engine/docs/reference_model_api.tex`	v0.1 stable	compiler-facing API
SW overview	`kv-cache-engine/docs/sw_overview.tex`	v0.1 stable	compiler-team orientation
TurboQuant+ paper	`turboquant/` (in progress)	draft, private	compression algorithm paper

2.4 Cross-block findings

Document	Source	Status	Role
KVCE × ACU integration audit	adaptive-precision-attention/ docs/findings/ kvce_acu_integration_audit.tex	v0.2 current	integration test + remediation
Cross-block conflict register	adaptive-precision-attention/ docs/findings/ kvce_acu_architectural_conflicts.tex	v0.1 (forthcoming)	standing design-tension register
Phase 1 entropy finding	adaptive-precision-attention/ docs/findings/ phase1-entropy-finding.md	frozen	evolutionary policy discovery
Sparsity controller finding	adaptive-precision-attention/ docs/findings/ sparsity-controller-finding.md	shelved	XAttention proxy negative result

2.5 Adjacent research

Document	Source	Status	Role
Don't Waste Bits FPGA controller	dont-waste-bits/research/ paper/fpga_controller_paper.tex	draft, submission-ready	variable-precision controller paper
DWB original (verified)	dont-waste-bits/2604.04722v1.pdf	external (CVPR 2026)	reference
LASSO KV coprocessor	lasso/writing_outputs/ lasso_paper.tex	draft	Sky130 KV-cache compressor
HADES step 5	decode-pipeline-optimizer/ paper/hades_step5.tex	draft	speculative decoding assist
FP4 multiplier	fp4-multiplier/paper.pdf	PDF only	minimum-gate FP4 (E2M1)
Chipathon microarchitecture	Chipathon/docs/ Microarchitecture.pdf	PDF only	SSCS Chipathon 130nm design

2.6 Tooling / adjacent

Repositories with no LaTeX documents but supporting the library's context: architecture/.github/, website, .github, SRAM_16384x32, kelle-simulator, kelle-fpga, kv-accelerator-comparison, OpenLane, openlane2, skywater-pdk, tiny-gpu. These appear in the library only as cross-reference targets, not as document sources.

3. Three structural options

3.1 Option A: dedicated LonghornSilicon/library repository

Create a new repository whose sole purpose is the canonical index. Contains a master index.tex, a shared references.bib, and a GitHub Pages render.

Pros

Clean separation of concerns; library has no other purpose so it cannot drift into being half-architecture, half-index. Discoverable as "the library" from the organization landing page.

Cons

Yet another repository to maintain. Risk of becoming a duplicate storage of papers rather than a pure index. Cross-references from existing repos must be added by hand.

3.2 Option B: extend the existing `architecture` repository

`LonghornSilicon/architecture` already exists, already holds the system-level λ paper, the dataflow walkthrough, the status file, and the floorplan visualisation. Add to it a `papers/` directory with the library index and shared bibliography.

Pros

Zero new repositories. The architecture repo's existing role (cross-block reference) generalises naturally to "cross-block documentation index". The λ paper already pulls together all block claims — the library is its natural sibling. GitHub Pages can render directly from this repository.

Cons

The architecture repo's scope widens; "architecture" now also means "publication index". Mitigated by clear directory boundaries (`paper/` for λ, `papers/` for the library index, `specs/` for cross-block specs).

3.3 Option C: distributed cross-references, no new home

Add a `LIBRARY.md` or `INDEX.md` to each repository that cross-links to the others. No central index.

Pros

Lowest effort. Each repo stays self-contained.

Cons

No single entry point for an external visitor. Cross-references must be kept in sync across nineteen repositories. Does not address the "which paper covers what" question for someone scanning from outside.

3.4 Recommendation

Option B: fold the library into `LonghornSilicon/architecture`. The architecture repo is already the de-facto cross-cutting home; the λ paper is already there; the dataflow walkthrough is already there; adding the index closes the loop without creating new repositories. Option A is the second-best choice if a clean separation of concerns becomes preferred later; Option C is rejected as not solving the external-visitor problem.

4. Proposed structure within `architecture`

```
LonghornSilicon/architecture/
|-- README.md                  # entry point, links to papers/INDEX
|-- STATUS.md                  # existing per-block development status
|-- arch.yml                   # existing structured architecture model
```

```
|-- dataflow_walkthrough.md    # existing dataflow explainer
|-- floorplan.html             # existing floorplan visualisation
|-- paper/
|   `-- lambda.tex             # existing system-level paper
|-- papers/                    # NEW: library
|   |-- INDEX.md               # human-readable index, rendered to Pages
|   |-- index.tex              # LaTeX-typeset companion (this proposal's
|   |                          #   document table, kept in sync)
|   |-- references.bib         # NEW: shared bibliography for all
|   |                          #   LonghornSilicon papers to cite
|   `-- abstracts/
|       |-- adaptive_precision_attention.md
|       |-- precision_controller_isa.md
|       |-- kv_cache_engine_isa.md
|       |-- turboquant.md
|       |-- fpga_controller_paper.md
|       |-- lasso_paper.md
|       |-- hades_step5.md
|       |-- kvce_acu_integration_audit.md
|       `-- ...
|-- specs/                     # NEW: convenience links to canonical
|   |                          #   block specs (symlinks or short files)
|   `-- README.md              # ``each block's ISA + ref model API''
`-- findings/                  # NEW: cross-block findings sourced from
    |                          #   block repos
    `-- README.md              # links to integration audit etc.
```

The `papers/` directory is the library proper. Each entry in `papers/abstracts/` is a one-page markdown summary (canonical title, authors, status, abstract, link to the authoritative source repo). The `INDEX.md` aggregates these into a single rendered page; `index.tex` is its LaTeX-typeset form for inclusion in funding reports or partnership packages.

5. Shared bibliography

The single highest-leverage artifact is `papers/references.bib`. Today, when the `adaptive_precision_attention` paper wants to cite TurboQuant+, the author hand-writes a BibTeX stanza. When the KVCE `lasso_paper` wants to cite the precision controller, another hand-written stanza appears, potentially with a different author list or title formatting. A canonical `references.bib` with stanzas like:

```
@misc{lhs_acu_precision_controller,
  title  = {Adaptive Precision Attention: Evolutionary
            Discovery to Hardware-Verified Ratio Gates},
  author = {Talasila, Chaithu and {LonghornSilicon}},
  year   = {2026},
  url    = {https://github.com/LonghornSilicon/
            adaptive-precision-attention},
  note   = {Block 1 of the LonghornSilicon LLM
            inference accelerator},
}
```

```
@misc{lhs_kvce_turboquant,
  title  = {TurboQuant+: Asymmetric K/V Compression
            for LLM KV Caches},
  ...
}
```

allows every paper in the organization to cite every other paper consistently. Future-us, advisors, foundry partners, and reviewers all encounter the same author lists and titles for the same work.

6. Implementation plan

Four steps, each landable in under thirty minutes by a single session.

1. **Bootstrap the `papers/` directory in `architecture`.** Add the directory, the `INDEX.md` seeded from §**??**, the `abstracts/` sub-directory with one stub abstract per document, and the empty `references.bib`.
2. **Populate abstracts.** For each document in §**??**, copy the existing abstract (or write one where missing) into the corresponding `papers/abstracts/<name>.md` file. Link to the canonical source.
3. **Populate the shared bibliography.** Add one BibTeX stanza per document to `references.bib`, with consistent author, title, and URL fields. Update existing papers to `\bibliography{../architecture/p` (or copy the bib into each paper as needed).
4. **Render to GitHub Pages.** Enable Pages on `architecture` pointing at `papers/INDEX.md`. The organization landing page links to it. Done.

Total work to first usable state: approximately two hours.

7. Open questions

1. Should TurboQuant+ live in `turboquant` (current home) or be promoted to a dedicated repository (e.g. `turboquant-plus`) now that it is the canonical KVCE compression algorithm?
2. Should the library index also enumerate *external* references the LonghornSilicon papers consistently cite (FlashAttention 1/2/3, GEAR, KIVI, KVQuant, RotateKV, SHIELD, etc.) so that the shared `references.bib` stops requiring hand-additions in each paper?
3. Does the website (`LonghornSilicon/website`) consume the library directly (e.g. render the index in-place) or maintain its own publication list that pulls from the library?
4. Status semantics: do we adopt three states (*draft*, *submitted*, *accepted*) or finer granularity (*draft / under-review / accepted / published / superseded / archived*)?
5. Versioning: do papers carry their own version codes (`pc-isa-0.1`, `kvce-isa-0.1`, `kvce-acu-audit-0.2`) into the library entries, or does the library normalise to a single versioning scheme?

These are not blockers; they can be resolved during step 2 of the implementation plan.

Change log

- `library-0.1` (2026-05-22): inventory completed across nineteen organization repositories; structural options evaluated; recommendation made; implementation plan drafted.